Lists

ISTA 130

Today

- ➔ Python list data type:
  - Creating lists
  - Indexing and slicing
  - Functions
  - Strings
  - Mutability
  - List copies
  - Methods
- ➔ One of the fundamental features of Python.

Last Time

➔ More strings.
➔ File I/O (input and output).

Opening And Reading Files

- The built-in **open** function opens a file:

```
open(<filename>, <mode>)
```

- `open` returns a file *object* that we use to read the file:

```
# open a file in 'read' mode and
# store the file object in a variable:
file_reader = open('somefile.txt', 'r')
```

- The `close` method tells a file object to release the file once we're done with it:

```
file_reader.close()
```

Reading Files with a `for` Loop

- Here's how we read a file line by line with a for-loop:

```
file_reader = open('example0.txt', 'r')

for line in file_reader:
    print(line)
    print('repr: ' + repr(line))
```

- What does this `for` loop remind you of?
  - it's similar to looping through every character in a string
- The for-loop iterates through each line in the file
  - On each iteration `Python` assigns a string to `line`
  - The string is the next single line of text from the file
  - When there are no more lines left the loop exits

Traversing a file by line

Remove the Newline

- The `print` function puts a newline (`'\n'`) at the end of each line it prints
- Each line in a text file is already ended with a newline.
- How can we remove the extra newline?
  - Use the `strip` method on each line we read in:

```
file = open('iwalktheline.txt', 'r')
for line in file:
    print(line.strip())
file.close()
```

Closing Files After Use

- So, `Python` might wait until the last minute to actually write things out to your file
- Closing your file causes `Python` to go ahead and write everything out

Python's List Data Type

- A *list* is a *sequence* of values (referred to as elements)
- You can create lists using [ ]'s

```
my_list = [] # an empty list
heights = [66, 59, 72]
names = ["Ed", "Tom", "Ann"]
stuff = [yertle, 34, math.pi, "egg yolk"]
```

Lists

To allow us to work more easily with large collections of values we'll introduce a new data type.

- We've seen several different *types* of values so far:

  `int`, `float`, `str`, `bool`

- The *list* type is a *sequence* of values
  - it allows us to store a collection of related values in one data structure
- We refer to the individual values in a list as *elements*

What other sequence type have we met?

Creating Lists

- To create a list *literal* we enclose a series of comma separated values in `[ ]`'s
- For example:

```
[3, 88, 0, 200, 14]
```

- As with any type of value, you can assign a list to a variable:

```
some_numbers = [3, 88, 0, 200, 14]
```

- The **print** function knows how to print a list:

```
print(some_numbers) → [3, 88, 0, 200, 14]
```

- Let's verify the *type*:

```
print(type(some_numbers)) → <class 'list'>
```

What Can You Put in a List?

You can put any type of value in a list.

- Strings:

```
["apples", "beer", "cheese", "duck"]
```

- Floats:

```
[2.7, 99.023, -4.31, 0.00000000001, 42.0]
```

- Booleans:

```
[True, x < 52, not False]
```

- Even Turtles!

```
[yertle, mertle, some_other_turtle, turtle.Turtle()]
```

What Can You Put in a List?

- The values do not have to be the same type:

```
[42, "beer", True, math.pi, -99, yertle]
```

- You can even put lists in lists:

```
[54, "beer", True, math.pi, [0, 1, 2, 3, 4]]
```

  - We refer to a list inside of a list as a "nested" list

- A list with no elements is called an "empty list": `[ ]`
  - (a string with no characters is called an "empty string".)

Python's List Data Type

```
mylist = [43, True, 'Rocket', [10, False, 'dog']]
# How many elements does mylist have?
# What types are they?
```

How do you think we might get
Python to tell us how many elements?

Using Lists

Using lists is similar to using strings in many ways (both are sequences):

- The `len` function works on lists:

```
some_numbers = [3, 88, 0, 200, 14]
print(len(some_numbers)) → 5
```

- The indexing operator works too:

```
print(some_numbers[0]) → 3
```

  - Here we're printing the 0th element of the list

Indexing into Lists

```
numbers = [3, 88, 0, 200, 14]
print(numbers[0]) → 3
print(numbers[1]) → 88
print(numbers[2]) → 0
print(numbers[3]) → 200
print(numbers[4]) → 14
print(numbers[5]) →
Traceback (most recent call last):
  File "/listExample1.py", line 16, in <module>
  IndexError: list index out of range
```

- Like strings, if we index outside of the list we get an error

Indexing into Lists

```
numbers = [3, 88, 0, 200, 14]
```

- Negative indexing works like it does for strings:

  `print(numbers[-1])` → 14

  `print(numbers[-2])` → 200

Indexing into Lists

```
food = ['apples', 'beer', 'cheese', 'duck']
```

- What's the difference between this:

```
len(food)
```

- and this?

```
len(food[0])
```

Indexing into Lists

```
food = ['apples', 'beer', 'cheese', 'duck']
```

- What's the difference between this:

```
len(food)
```

- and this?

```
len(food[0])
```

- Python evaluates the first one like this:

```
len(food)
len(['apples', 'beer', 'cheese', 'duck'])
4
```

- and the second one like this:

```
len(food[0])
len(['apples', 'beer', 'cheese', 'duck'][0])
len('apples')
6
```

Indexing into Lists

```
food = ['apples', 'beer', 'cheese', 'duck']
```

- What do you think this will do?

```
print(food[0][0])
```

- If you pass a *list* to the `len` function it returns the number of *elements* in the list
- If you pass a *string* to the `len` function it returns the number of *characters* in the string

Indexing into Lists

```
food = ['apples', 'beer', 'cheese', 'duck']
```

- What do you think this will do?

```
print(food[0][0])
```

- Python evaluates it like this:

```
print(food[0][0])
print(['apples', 'beer', 'cheese', 'duck'][0][0])
print('apples'[0])
print('a')
```

- If you pass a *list* to the `len` function it returns the number of *elements* in the list
- If you pass a *string* to the `len` function it returns the number of *characters* in the string

Slicing Lists

- You can slice a list:

```
food_list = ["apples", "beer", "cheese", "duck"]
print(food_list[1:3]) → ["beer", "cheese"]
print(food_list[2:])  → ["cheese", "duck"]
```

- When you sliced a string the result was a *new* string
- When you slice a list the result is a *new* list

```
a = food_list[1:2]
print(a) → ["beer"]

b = food_list[1:1]
print(b) → []
```

- The original list remains unchanged
  - `print(food)` →
    `["apples", "beer", "cheese", "duck"]`

Indexing And Slicing

- Indexing and slicing lists:

```
food_list = ["apples", "beer", "cheese", "duck"]
food_list[0] → "apples"
food_list[1] → "beer"
food_list[1:3] → ["beer", "cheese"]
food_list[2:] → ["cheese", "duck"]
```

`[ ]`

- `[]`, `{}`, and `()` all do double duty in Python.
- `[]` is the indexing operator, the slicing operator, and the list literal delimiter.
- `[]` work like the quotes around a string literal.
- What will this do: `[12, 23, 44][1]`?

`[]` and Building Lists

- `[]` is also the empty list.
- We can use it to build lists like we used 0 to accumulate values and the empty string to build strings.

Concatenating and Repeating

- Concatenation: `A + B`
  - (make a *new* list containing all elements from `A` and `B`)

```
['elk', 'moose'] + ['bee', 'fox'] →
    ['elk', 'moose', 'bee', 'fox']
```

- Repetition: `A * n`
  - (make a *new* list containing `n` copies of the elements in `A`)

```
['elk', 'bee'] * 3 →
    ['elk', 'bee', 'elk', 'bee', 'elk', 'bee']

[2, 4, 6] * 2 →
    [2, 4, 6, 2, 4, 6]
```

Never do this: `[[]] * 3`.

Do this instead: `[[], [], []]`.

Traversing Lists by Element

- Similar to strings, you can *traverse* a list element *by element* with a for-loop:

```
numbers = [1, 2, 3, 4]
for n in numbers:
    print(n)
```

Output:

```
1
2
3
4
```

Traversing Lists by Element

- What happens when we do this:

```
numbers = [1, 2, 3, 4]
for n in numbers:
    n = "mud"
print(numbers)
print(n)
```

Traversing Lists by Element

- What happens when we do this:

```
numbers = [1, 2, 3, 4]
for n in numbers:
    n = "mud"
print(numbers)
print(n)
```

- `n` is just a variable. You can assign a value to it (but you usually don't want to).
- Output:

```
[1, 2, 3, 4]
mud
```

- Assigning to the variable `n` just puts a new value inside `n`, it has no effect on the list

Traversing Lists by Index

- You can also *traverse* a list *by index value* with a for-loop:

```
numbers = [1, 2, 3, 4]
for i in range(len(numbers)):
    print(numbers[i])
```

- Output:

```
1
2
3
4
```

Traversing Lists by Index

- Lists are mutable. You can use an index to change the contents of a list:

```
numbers = [1, 2, 3, 4]
for i in range(len(numbers)):
    numbers[i] = "mud"
print(numbers)
```

Traversing Lists by Index

- Lists are mutable. You can use an index to change the contents of a list:

```
numbers = [1, 2, 3, 4]
for i in range(len(numbers)):
   numbers[i] = "mud"
print(numbers)
```

- Output:

```
['mud', 'mud', 'mud', 'mud']
```

- Assigning to `numbers[i]` changes (mutates) the list
  - we're using `i` to access and alter the contents of the list found in the variable `numbers`

Functions `len`, `min`, `max`, `sum`

Some useful functions that accept lists as parameters:

- **len** returns the length of a sequence (list, string):

```
len([1, 2, 3, 4]) → 4
len([]) → 0
```

- **min** returns the smallest value

```
min([1, 2, 3, 4]) → 1
min(['c', 'd', 'a']) → 'a'
```

- **max** returns the largest value

```
max([1, 2, 3, 4]) → 4
max(['c', 'd', 'a']) → 'd'
```

- **sum** returns the sum of a list of numbers

```
sum([1, 2, 3, 4]) → 10
```

Using Lists

- A nested list is a *single* element:

```
things = [54, "beer", math.pi, [10, 20, 30]]

print(len(things)) → 4
print(len(things[3])) → 3
print(len(things[1])) → 4
```

- The same list with the nested list as a variable:

```
nums = [10, 20, 30]
things = [54, "beer", math.pi, nums]
print(len(things)) → 4
print(len(things[3])) → 3
```

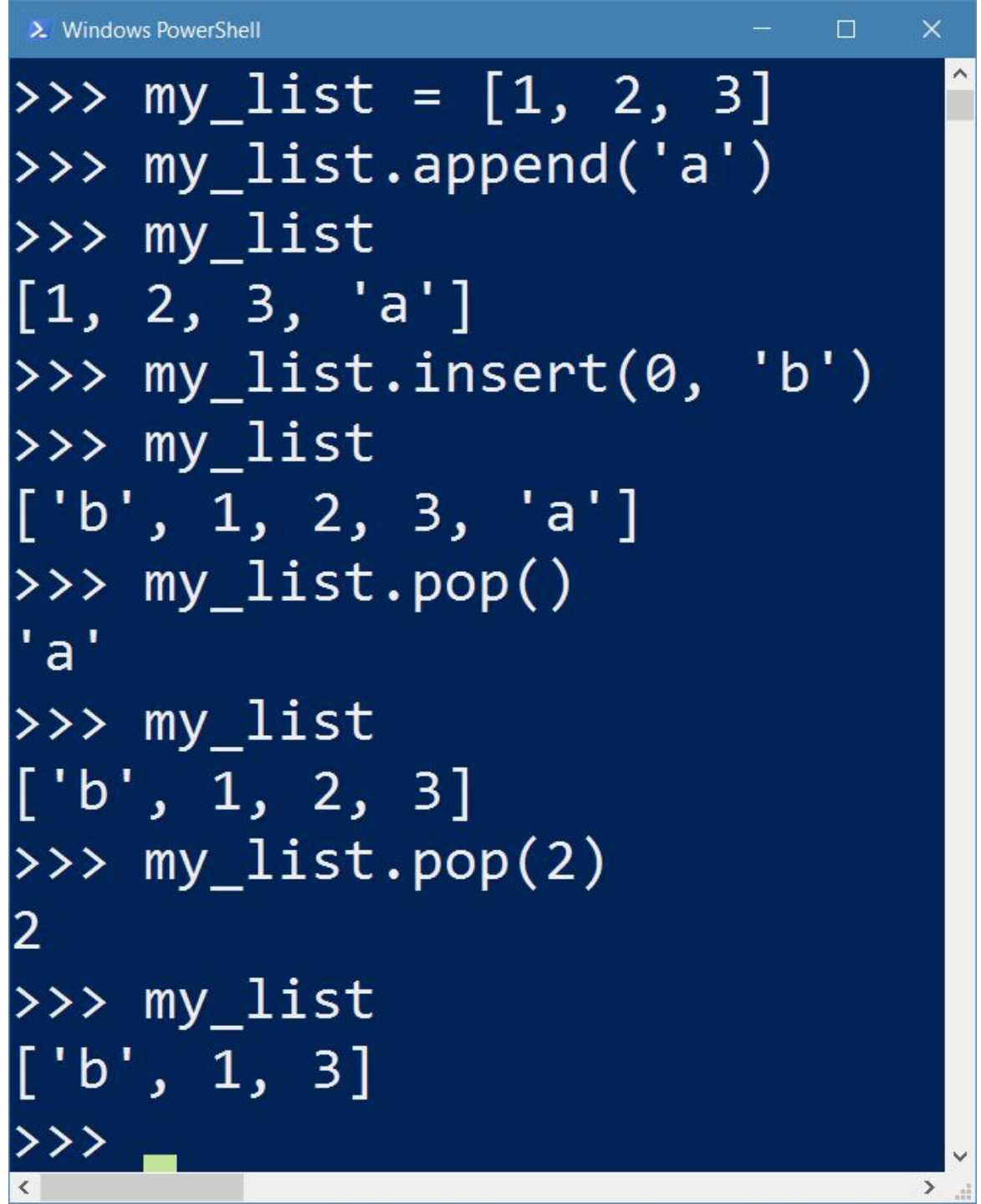
Using Lists

- What about this?

```
stuff = [10, 'burp', yertle]
things = [54, 'beer', math.pi, stuff]
print(len(things)) → 4
print(len(things[3])) → 3
print(things[3][1][0]) → 'b'
```

- Evaluate it step by step:

```
print(things[3][1][0])
print([54, 'beer', math.pi, stuff][3][1][0])
print(stuff[1][0])
print([10, 'burp', yertle][1][0])
print('burp'[0])
print('b')
```


Windows PowerShell
>>> my_list = [1, 2, 3]
>>> my_list.append('a')
>>> my_list
[1, 2, 3, 'a']
>>> my_list.insert(0, 'b')
>>> my_list
['b', 1, 2, 3, 'a']
>>> my_list.pop()
'a'
>>> my_list
['b', 1, 2, 3]
>>> my_list.pop(2)
2
>>> my_list
['b', 1, 3]
>>>